SKILL Week 10

Task1 : To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events

Aim : To register signal handlers, configure signal actions, handle interrupts, manage signal context, verify program stability, and test different signal events.

Step 1: Create the C program

nano sigaction_demo.c

```
  GNU nano 7.2                          sigaction_demo.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

void signal_handler(int signal_number, siginfo_t *info, void *context)
{
    printf("\nSignal received: %d\n", signal_number);

    if (info != NULL)
    {
        printf("Signal sent by PID: %d\n", info->si_pid);
    }

    if (context != NULL)
    {
        printf("Signal context received successfully.\n");
    }

    if (signal_number == SIGINT)
    {
        printf("SIGINT handled safely.\n");
    }
    else if (signal_number == SIGTERM)
    {
        printf("SIGTERM handled safely.\n");
    }
}
```

```c
int main()
{
    struct sigaction action;

    action.sa_sigaction = signal_handler;
    sigemptyset(&action.sa_mask);
    action.sa_flags = SA_SIGINFO;

    if (sigaction(SIGINT, &action, NULL) == -1)
    {
        perror("sigaction SIGINT failed");
        return 1;
    }

    if (sigaction(SIGTERM, &action, NULL) == -1)
    {
        perror("sigaction SIGTERM failed");
        return 1;
    }

    printf("Signal handler registered successfully.\n");
    printf("Process PID: %d\n", getpid());
    printf("Press Ctrl+C to test SIGINT.\n");
    printf("Use 'kill -TERM PID' to test SIGTERM.\n");

    while (1)
    {
        printf("Program is running safely...\n");
        sleep(3);
    }

    return 0;
}
```

Step 2 — Compile and run the program

```
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ nano sigaction_demo.c
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ gcc sigaction_demo.c -o sigaction_demo
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ ./sigaction_demo
Signal handler registered successfully.
Process PID: 780
Press Ctrl+C to test SIGINT.
Use 'kill -TERM PID' to test SIGTERM.
Program is running safely...
^C
Signal received: 2
Signal sent by PID: 0
Signal context received successfully.
SIGINT handled safely.
Program is running safely...
Program is running safely...
```

```
alurusaranya@LAPTOP-0PA61QEN:~$ pgrep sigaction_demo
780
alurusaranya@LAPTOP-0PA61QEN:~$ kill -TERM 780
alurusaranya@LAPTOP-0PA61QEN:~$ pgrep sigaction_demo
780
alurusaranya@LAPTOP-0PA61QEN:~$ kill -INT 780
alurusaranya@LAPTOP-0PA61QEN:~$
```

```
Signal received: 15
Signal sent by PID: 791
Signal context received successfully.
SIGTERM handled safely.
Program is running safely...
```

```
Signal received: 2
Signal sent by PID: 791
Signal context received successfully.
SIGINT handled safely.
```

Step 3: Test invalid input

```
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ ./job_manager
1. Create background job
2. List jobs
3. Exit
Enter choice: 5
Invalid choice.
1. Create background job
2. List jobs
3. Exit
Enter choice: 3
Exiting job manager.
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$
```

Observation:

The program successfully registered signal handlers using the sigaction() system call. Signal actions were configured for SIGINT and SIGTERM using the sigaction structure and the SA_SIGINFO flag. When signals were generated using Ctrl+C and the Linux kill command, the registered handler was invoked without terminating the program. The siginfo_t structure was used to obtain information about the process that sent the signal, and the signal context was received successfully. The program continued running after handling the signals, demonstrating stable signal processing. SIGKILL was used separately to terminate the test program because it cannot be caught or handled.

Result:

Signal handlers were successfully registered and configured using sigaction(). Interrupts and signal events were handled correctly, signal context information was accessed, and program stability was verified through multiple signal tests.

Task 2 : . Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling

Aim : This task demonstrates how a shell handles Ctrl+C (SIGINT) without terminating the shell itself, while forwarding the signal to the foreground job.

Step 1: Create the C program

nano sigint_forward.c

```
  GNU nano 7.2                          sigint_forward.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h>

pid_t foreground_pid = -1;

void handle_sigint(int signal)
{
    printf("\nShell received SIGINT.\n");

    if (foreground_pid > 0)
    {
        printf("Forwarding SIGINT to foreground job (PID = %d)\n",
               foreground_pid);

        kill(foreground_pid, SIGINT);
    }
    else
    {
        printf("No foreground job is running.\n");
    }
}

int main()
{
    struct sigaction action;
    int status;

    action.sa_handler = handle_sigint;
    sigemptyset(&action.sa_mask);
    action.sa_flags = 0;

    if (sigaction(SIGINT, &action, NULL) == -1)
    {
        perror("sigaction failed");
        return 1;
    }
```

GNU nano 7.2 sigint_forward.c *

```c
    printf("Simple shell started.\n");
    printf("Shell PID: %d\n", getpid());

    while (1)
    {
        printf("\nshell> ");
        fflush(stdout);

        printf("Starting foreground job...\n");

        foreground_pid = fork();

        if (foreground_pid < 0)
        {
            perror("fork failed");
            return 1;
        }

        if (foreground_pid == 0)
        {
            printf("Foreground job started. PID = %d\n", getpid());

            while (1)
            {
                printf("Foreground job is running...\n");
                sleep(2);
            }

            exit(0);
        }

        printf("Foreground job PID: %d\n", foreground_pid);
        printf("Press Ctrl+C to interrupt the foreground job.\n");

        waitpid(foreground_pid, &status, 0);

        if (WIFSIGNALED(status))
        {
            printf("Foreground job terminated by signal %d.\n",
                   WTERMSIG(status));
        }

        printf("Job state updated: TERMINATED\n");

        foreground_pid = -1;

        printf("Shell is still running.\n");

        break;
    }

    printf("\nShell exiting normally.\n");

    return 0;
}
```

Step 2: Compile and run the program

```
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ nano sigint_forward.c
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ gcc sigint_forward.c -o sigint_forward
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ ./sigint_forward
Simple shell started.
Shell PID: 871

shell> Starting foreground job...
Foreground job PID: 872
Press Ctrl+C to interrupt the foreground job.
Foreground job started. PID = 872
Foreground job is running...
^C
Shell received SIGINT.
No foreground job is running.
Foreground job is running...

Shell received SIGINT.
Forwarding SIGINT to foreground job (PID = 872)
Job state updated: TERMINATED
Shell is still running.

Shell exiting normally.

Shell received SIGINT.
No foreground job is running.
Foreground job is running...
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ Foreground job is running...
```

Observation

The program successfully demonstrated SIGINT handling in a simple shell. When Ctrl+C was pressed, the shell captured the SIGINT signal using sigaction() instead of terminating itself. The shell forwarded the SIGINT signal to the foreground job using kill(). The foreground job terminated, and waitpid() was used to detect its termination. The job state was then updated to TERMINATED, while the shell remained active.

Result

The shell successfully captured and forwarded SIGINT to the foreground job. The foreground job was terminated correctly, the job state was updated, and the shell process remained protected from termination.

Task 3: . Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow

Aim : We can do this as a simple shell job-control simulation. The shell captures SIGTSTP (Ctrl+Z), sends it to the foreground process, records the job as STOPPED, and later resumes it using SIGCONT.

Step 1: Create a C file

nano sigtstp_job.c

```
  GNU nano 7.2                                  sigtstp_job.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

pid_t foreground_pid = -1;
char job_state[20] = "RUNNING";

void handle_sigtstp(int signal)
{
    if (foreground_pid > 0)
    {
        printf("\nShell received SIGTSTP (Ctrl+Z).\n");

        kill(foreground_pid, SIGSTOP);

        strcpy(job_state, "STOPPED");

        printf("SIGSTOP sent to foreground job.\n");
        printf("Job state updated: STOPPED\n");
    }
}

int main()
{
    struct sigaction action;
    int status;

    action.sa_handler = handle_sigtstp;
    sigemptyset(&action.sa_mask);
    action.sa_flags = 0;

    if (sigaction(SIGTSTP, &action, NULL) == -1)
    {
        perror("sigaction failed");
        return 1;
    }

    printf("Simple shell started.\n");
    printf("Shell PID: %d\n", getpid());

    foreground_pid = fork();

    if (foreground_pid < 0)
    {
        perror("fork failed");
        return 1;
    }
```

```
  GNU nano 7.2                                          sigtstp_job.c *
        return 1;
    }

    if (foreground_pid == 0)
    {
        signal(SIGTSTP, SIG_DFL);

        printf("Foreground job started. PID = %d\n", getpid());

        while (1)
        {
            printf("Foreground job is running...\n");
            sleep(2);
        }
    }

    printf("Foreground job PID: %d\n", foreground_pid);
    printf("Job state: %s\n", job_state);

    printf("\nPress Ctrl+Z to stop the foreground job.\n");

    waitpid(foreground_pid, &status, WUNTRACED);

    if (WIFSTOPPED(status))
    {
        strcpy(job_state, "STOPPED");

        printf("\nJob has been stopped.\n");
        printf("Job state: %s\n", job_state);
    }

    printf("\nPress Enter to resume the job using SIGCONT...\n");
    getchar();

    printf("Sending SIGCONT to foreground job...\n");

    kill(foreground_pid, SIGCONT);

    strcpy(job_state, "RUNNING");

    printf("Job state updated: %s\n", job_state);

    sleep(5);

    printf("\nTerminating the job...\n");

    kill(foreground_pid, SIGTERM);

    waitpid(foreground_pid, &status, 0);

    strcpy(job_state, "COMPLETED");

    printf("Final Job State: %s\n", job_state);
    printf("Shell is still running.\n");

    return 0;
}
```

Step 2: Compile and run the program

```
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ nano sigtstp_job.c

alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ gcc sigtstp_job.c -o sigtstp_job
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ ./sigtstp_job
Simple shell started.
Shell PID: 941
Foreground job PID: 942
Job state: RUNNING

Press Ctrl+Z to stop the foreground job.
Foreground job started. PID = 942
Foreground job is running...
^Z
Shell received SIGTSTP (Ctrl+Z).
SIGSTOP sent to foreground job.
Job state updated: STOPPED

Press Enter to resume the job using SIGCONT...

Sending SIGCONT to foreground job...
Job state updated: RUNNING
Foreground job is running...
Foreground job is running...
Foreground job is running...

Terminating the job...
Final Job State: COMPLETED
Shell is still running.
```

Observation

The program successfully simulated shell job control using SIGTSTP and SIGCONT. A foreground process was created using fork(). When Ctrl+Z was pressed, the shell captured SIGTSTP and sent SIGSTOP to the foreground process. The job state was updated from RUNNING to STOPPED. Later, SIGCONT was sent to resume the stopped process, and the job state changed back to RUNNING. The process was then terminated and the final job state was updated to COMPLETED.

Result

The SIGTSTP suspension and SIGCONT resume workflow was successfully implemented and tested. The shell was able to capture the suspension request, maintain the stopped job state, resume the process later, and update the job state correctly.

Task-4 : Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior

Aim : This program demonstrates basic job-control process groups. The parent acts as the shell, creates a child process group, transfers terminal control to that group, sends a signal to the group, and then restores terminal control to the shell.

Step 1: Create a C file

nano process_group.c

```
 GNU nano 7.2                                  process_group.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <termios.h>

int main()
{
    pid_t pid;
    pid_t shell_pgid;
    pid_t child_pgid;
    int status;

    /*
     * Ignore SIGTTOU so the shell can safely
     * change terminal ownership.
     */
    signal(SIGTTOU, SIG_IGN);

    shell_pgid = getpgrp();

    printf("Shell PID: %d\n", getpid());
    printf("Shell PGID: %d\n", shell_pgid);

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        setpgid(0, 0);

        child_pgid = getpgrp();

        printf("\nChild process created.\n");
        printf("Child PID: %d\n", getpid());
        printf("Child PGID: %d\n", child_pgid);

        while (1)
        {
            printf("Child process group is running...\n");
            sleep(2);
        }
    }
```

```
 GNU nano 7.2                                    process_group.c *

    setpgid(pid, pid);

    child_pgid = pid;

    printf("\nChild process group created.\n");
    printf("Child PID: %d\n", pid);
    printf("Child PGID: %d\n", child_pgid);

    sleep(2);

    printf("\nTransferring terminal control to child group...\n");

    if (tcsetpgrp(STDIN_FILENO, child_pgid) == -1)
    {
        perror("tcsetpgrp failed");
    }
    else
    {
        printf("Terminal control transferred to PGID %d.\n",
               child_pgid);
    }

    sleep(2);

    printf("\nSending SIGTERM to child process group...\n");

    if (kill(-child_pgid, SIGTERM) == -1)
    {
        perror("kill failed");
    }
    else
    {
        printf("SIGTERM sent to process group %d.\n",
               child_pgid);
    }

    waitpid(pid, &status, 0);

    printf("\nRestoring terminal control to shell...\n");

    if (tcsetpgrp(STDIN_FILENO, shell_pgid) == -1)
    {
        perror("tcsetpgrp restore failed");
    }
    else
    {
        printf("Terminal control restored to shell PGID %d.\n",
               shell_pgid);
    }

    printf("\nChild process group terminated.\n");
    printf("Shell continues normally.\n");

    return 0;
}
```

Step 2: Compile and run the program

```
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ nano process_group.c
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ gcc process_group.c -o process_group
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ ./process_group
Shell PID: 988
Shell PGID: 988

Child process group created.
Child PID: 989
Child PGID: 989

Child process created.
Child PID: 989
Child PGID: 989
Child process group is running...

Transferring terminal control to child group...
Child process group is running...
Terminal control transferred to PGID 989.
Child process group is running...

Sending SIGTERM to child process group...
SIGTERM sent to process group 989.

Restoring terminal control to shell...
Terminal control restored to shell PGID 988.

Child process group terminated.
Shell continues normally.
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$
```

Observation

The program successfully demonstrated basic job-control process groups. The parent process acted as the shell and created a child process using fork(). The child was assigned to a separate process group using setpgid(), and the process group ID was obtained using getpgrp(). Terminal control was transferred to the child process group using tcsetpgrp(). A SIGTERM signal was sent to the complete child process group using kill() with a negative process group ID. After the child terminated, terminal control was restored to the shell. SIGTTOU was ignored to allow safe terminal-control operations.

Result

Process groups were successfully created and managed. Terminal control was transferred to and restored from the child process group, signals were coordinated using the process group ID, and the shell continued normally after the child process group terminated.